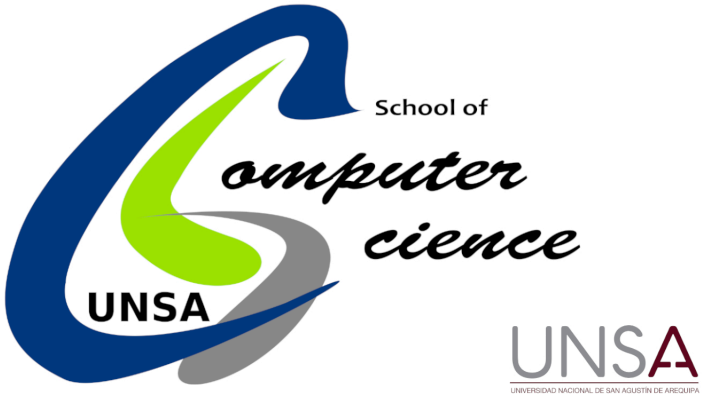
UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS



ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN
COMPUTACION PARALELA Y DISTRIBUIDA

LABORATORIO 5: PROGRAMACIÓN CON PTHREADS: CONCURRENCIA Y SINCRONIZACIÓN

INTEGRANTES:

- Lopez Zegarra, Ivan Alexander

DOCENTE:

ALVARO HENRY MAMANI ALIAGA

AREQUIPA – PERÚ
2026

Índice

1. Introducción 1
2. Entorno de Ejecución y Características del Equipo 1
3. Parte I: Cálculo de π 2
   - 3.1. Metodología e Implementación 2
   - 3.2. Análisis de Variables y Sección Crítica 3
   - 3.3. Evaluación Gráfica de Rendimiento, Speedup y Eficiencia 3
   - 3.4. Preguntas Teóricas y Comportamiento del Rendimiento 4
4. Parte II: Multiplicación Matriz-Vector 5
   - 4.1. Metodología de Descomposición 1D 5
   - 4.2. Justificación Técnica de Ausencia de Mutex 6
   - 4.3. Crecimiento del Speedup y Cuello de Botella de Memoria 7
5. Parte III: Lista Enlazada Multithreading 7
   - 5.1. Escenario y Variantes de Sincronización 7
   - 5.2. Condiciones de Carrera e Inconsistencias en Lista sin Sincronización 8
   - 5.3. Granularidad de Bloqueo: Mutex vs. RWLock 9
6. Parte IV: Implementaciones de Barreras 9
   - 6.1. Variantes de Sincronización Multifase 9
   - 6.2. Análisis Comparativo de Sobrecarga y Saturación 10
7. Resultados y Análisis 11
   - 7.1. Tablas Consolidadas de Datos Empíricos 11
   - 7.2. Figuras de Eficiencia y Escalabilidad en Matriz y Lista 12
8. Problemas Encontrados y Soluciones 13
9. Conclusiones 14

1. Introducción

La programación paralela en arquitecturas de memoria compartida mediante la biblioteca estándar POSIX Threads (`Pthreads`) constituye uno de los pilares fundamentales de la computación de alto rendimiento. En este modelo, múltiples hilos de ejecución cooperan dentro del mismo espacio de direcciones virtuales, permitiendo un intercambio de información de mínima latencia a través de la memoria principal. No obstante, esta compartición irrestricta de memoria introduce desafíos inherentes a la concurrencia: condiciones de carrera (*race conditions*), inconsistencias de datos, bloqueos mutuos (*deadlocks*), inanición (*starvation*) y sobrecargas por contención y cambios de contexto.

El presente informe documenta el desarrollo, análisis experimental y evaluación de escalabilidad del **Laboratorio 5: Programación con Pthreads: Concurrencia y Sincronización**. La práctica se estructura en cuatro actividades centrales que exploran los fundamentos y trade-offs de la programación concurrente:

1. **Cálculo de π mediante series alternadas:** Paralelización balanceada de un cómputo intensivo en CPU (*CPU-bound*), reducción con acumuladores locales y protección de sección crítica vía `pthread_mutex`.
2. **Multiplicación Matriz-Vector:** Descomposición unidimensional por bloques de filas contiguas, evaluando el acceso a memoria disjunto y los límites de ancho de banda del subsistema de memoria (*memory-bound*).
3. **Estructura de Datos Concurrente (Lista Enlazada):** Evaluación comparativa entre una versión sin sincronización (que evidencia los riesgos de punteros corruptos y ciclos), un candado global exclusivo (`pthread_mutex_t`) y candados de lectura/escritura (`pthread_rwlock_t`).
4. **Sincronización Colectiva Multifase (Barreras):** Implementación y comparación de tres variantes clásicas: espera activa (*busy-waiting*), semáforos POSIX y variables de condición de Pthreads.

El código fuente completo de todas las implementaciones, los scripts de automatización de benchmarks, los datos experimentales y la infraestructura de generación de figuras se encuentran disponibles públicamente en el siguiente repositorio de GitHub:

Repositorio del Proyecto:
https://github.com/Iv4n0f/paralela-lab5

2. Entorno de Ejecución y Características del Equipo

Las mediciones empíricas se llevaron a cabo en el entorno de ejecución local nativo sobre el sistema operativo Linux. La microarquitectura y topología del procesador fueron verificadas exhaustivamente mediante la utilidad `lscpu`.

Parámetro del Sistema	**Especificación de Hardware / Software**
Procesador	AMD Ryzen 7 5700U with Radeon Graphics
Microarquitectura	AMD Zen 2 (Lucienne), 7 nm
Núcleos Físicos	8 núcleos físicos
Hilos Lógicos (SMT)	16 hilos mediante Simultaneous Multithreading
Frecuencia Base / Boost	1.80 GHz base / hasta 4.37 GHz boost dinámico
Memoria Caché L1d / L1i	8 × 32 KiB (256 KiB L1d) / 8 × 32 KiB (256 KiB L1i)
Memoria Caché L2	8 × 512 KiB (4 MiB L2 dedicado por núcleo)
Memoria Caché L3	2 complejos CCX × 4 MiB (8 MiB L3 unificado)
Memoria Principal (RAM)	16 GB DDR4-3200 MT/s Dual-Channel
Sistema Operativo	Linux 6.8.0 x86_64
Compilador C	GCC 13.x con flags `-O3 -Wall -Wextra -pthread`
Herramienta de Tipografía	Typst 0.15.1

Tabla 1: Características del entorno experimental de hardware y software.

El procesador dispone de dos umbrales arquitecturales cruciales para el análisis de escalabilidad:

1. **Región de Paralelismo Puro ($p \in [1, 8]$):** Cada hilo se mapea a un núcleo físico independiente, obteniendo acceso exclusivo a unidades funcionales (ALU, FPU) y cachés L1/L2.
2. **Región de Hyper-Threading / SMT ($p \in [9, 16]$):** Dos hilos compiten por las estructuras internas y puertos de ejecución del mismo núcleo físico.
3. **Región de Sobrecarga y Saturación ($p \in [17, 24]$):** La cantidad de hilos excede los núcleos lógicos del procesador, forzando al planificador del kernel (`CFS`) a multiplexar tiempo de CPU, lo que genera sobrecarga por cambios de contexto, expulsión de líneas de caché y contención en bloqueos activos.

Cada configuración se ejecutó un total de **4 repeticiones independientes**, descartando variaciones por frecuencia dinámica y promediando los tiempos de pared cronometrados estrictamente en la sección paralela con `clock_gettime(CLOCK_MONOTONIC, ...)`.

3. Parte I: Cálculo de π

3.1. Metodología e Implementación

Se aproximó el valor de π evaluando la serie alternada de Gregory-Leibniz:

$$\pi = 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1}$$

En la versión secuencial (`pi_secuencial.c`), un bucle simple acumula los términos de manera monótona. En la versión paralela (`pi_pthreads.c`), el espacio de iteraciones N se particiona equitativamente en bloques continuos disjuntos entre los p hilos disponibles.

Para garantizar un balanceo de carga óptimo cuando N no es múltiplo exacto de p, se calculó el cociente $n_{\text{base}} = \left\lfloor \frac{N}{p} \right\rfloor$ y el residuo $r = N \bmod p$. Los primeros r hilos reciben $n_{\text{base}} + 1$ iteraciones, mientras que los restantes $p - r$ hilos reciben n_{base}:

$$\text{my_first} = \begin{cases} \text{rank} \times (n_{\text{base}} + 1) & \text{si} \quad \text{rank} < r \\ r \times (n_{\text{base}} + 1) + (\text{rank} - r) \times n_{\text{base}} & \text{si} \quad \text{rank} \geq r \end{cases}$$

$$\text{my_last} = \text{my_first} + \begin{cases} n_{\text{base}} + 1 & \text{si} \quad \text{rank} < r \\ n_{\text{base}} & \text{si} \quad \text{rank} \geq r \end{cases}$$

3.2. Análisis de Variables y Sección Crítica

- **Variables Privadas:** Cada hilo mantiene en su pila privada su identificador de rango (`my_rank`), los índices de inicio y fin (`my_first`, `my_last`), el factor de alternancia de signo (`factor`) y, crucialmente, un acumulador local (`my_sum`).
- **Variables Compartidas:** El número total de términos (`n_terms`), la cantidad de hilos (`thread_count`), el acumulador final (`global_sum`) y el cerrojo de exclusión mutua (`mutex_sum`).
- **Resolución de la Sección Crítica:** En lugar de invocar `pthread_mutex_lock` en cada iteración —lo cual destruiría el rendimiento por contención extrema— cada hilo acumula millones de operaciones en su variable local privada `my_sum`. La sección crítica compartida se accede **exactamente una sola vez por hilo al finalizar su cómputo**, acumulando `global_sum += my_sum;`. Con ello, la sobrecarga de sincronización se reduce a un costo constante mínimo $O(p)$.

3.3. Evaluación Gráfica de Rendimiento, Speedup y Eficiencia

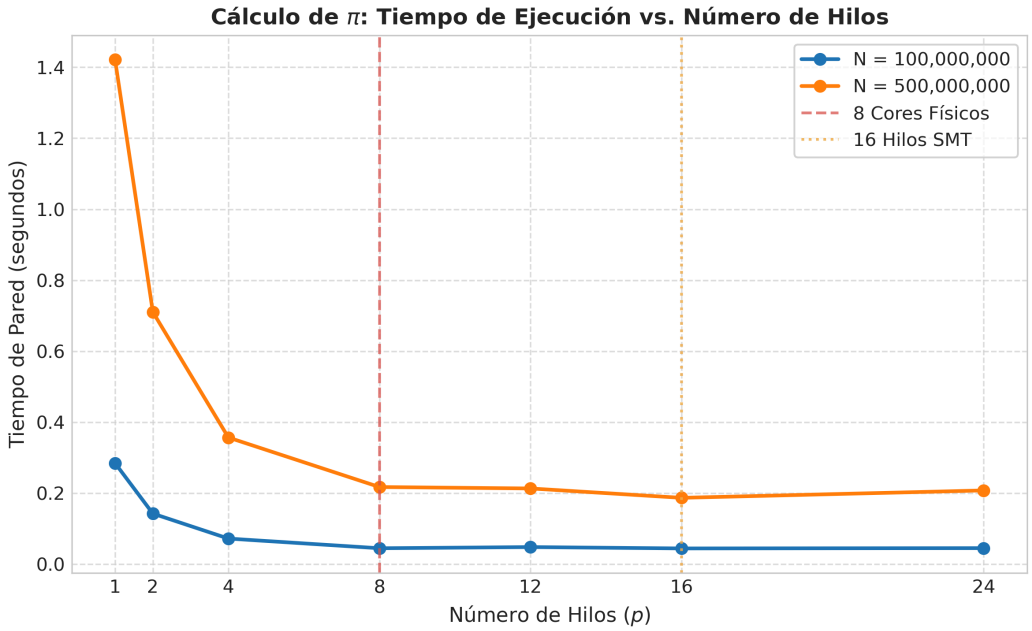


Figura 1: Tiempo de ejecución del cálculo de π vs. número de hilos (p) para $N = 10^8$ y $N = 5 \times 10^8$.

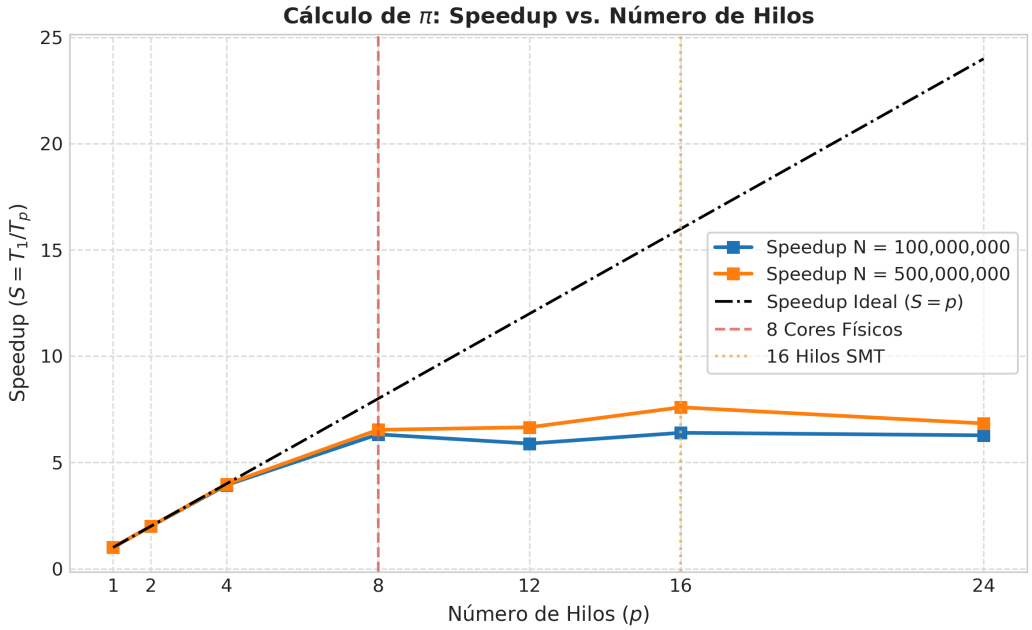


Figura 2: Speedup relativo ($S = \frac{T_1}{T_p}$) en el cálculo de π comparado con la línea de Speedup ideal ($S = p$).

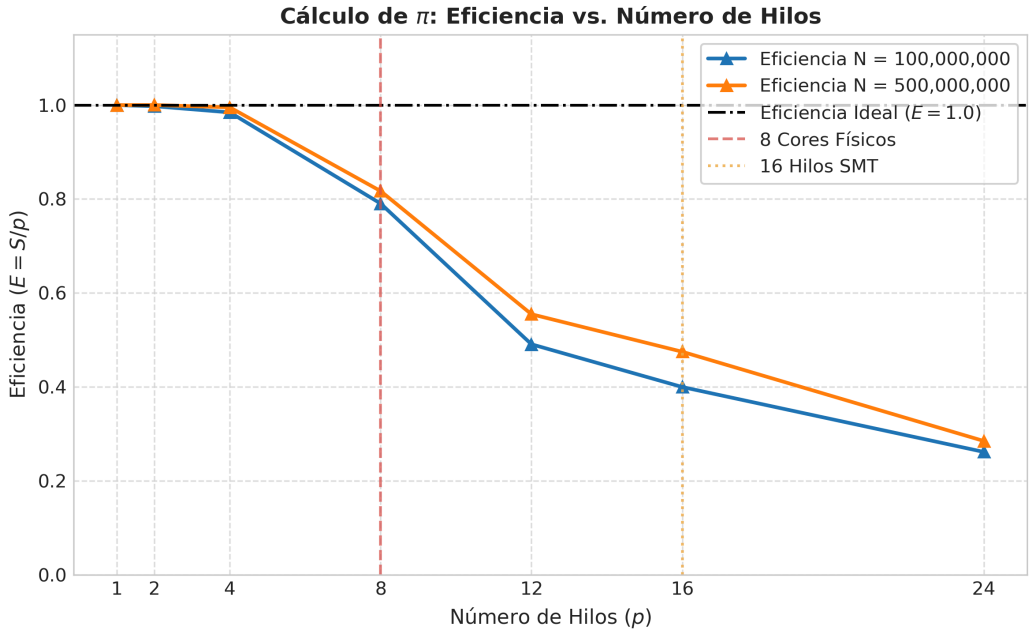


Figura 3: Eficiencia de cómputo ($E = \frac{S}{p}$) en el cálculo de π, con marcas verticales en 8 y 16 hilos.

3.4. Preguntas Teóricas y Comportamiento del Rendimiento

¿Cómo se distribuyó el trabajo?

El dominio discreto de iteraciones $[0, N-1]$ se descompuso en bloques contiguos balanceados. Al asignar bloques contiguos en lugar de un esquema cíclico, se maximiza la localidad temporal en los registros del

CPU y se simplifica el cálculo directo del signo del primer término $(-1)^{\text{my_first}}$ sin dependencias entre hilos.

¿Qué variables son compartidas y cuáles privadas?
Tal como se detalló, son compartidas `n_terms`, `thread_count`, `global_sum` y `mutex_sum`. Son estrictamente privadas a cada hilo `my_rank`, `my_first`, `my_last`, `factor` y `my_sum`.

¿Cómo se resolvió la sección crítica?
Se resolvió mediante un acumulador privado por hilo y una reducción única protegida por `pthread_mutex_t`. Esta técnica desacopla el progreso de los hilos durante el 99.999% de su tiempo de ejecución, eliminando la contención de buses.

¿Qué comportamiento exhibe el tiempo al aumentar hilos y por qué deja de mejorar el rendimiento?
Como se aprecia en las figuras de tiempo, speedup y eficiencia:

1. **Escalamiento cuasi-lineal ($1 \leq p \leq 4$):** Con $N = 500\text{M}$, el speedup alcanza $S = 2.00\times$ ($p = 2$) y $S = 3.98\times$ ($p = 4$), con una eficiencia prácticamente perfecta ($E \approx 1.00$).
2. **Región de 8 núcleos físicos ($p = 8$):** El speedup asciende a $S = 6.54\times$ ($E = 0.82$). La eficiencia disminuye ligeramente debido a la limitación térmica de la frecuencia boost multinúcleo.
3. **Rango de SMT / Hyper-Threading ($8 < p \leq 16$):** Al pasar a 12 y 16 hilos, el speedup alcanza su valor máximo de $S = 7.60\times$ ($p = 16$). Los núcleos lógicos SMT aprovechan las latencias de ejecución en los pipelines, pero la eficiencia se reduce a $E = 0.47$ dado que los núcleos físicos siguen siendo 8.
4. **Región de sobresuscripción ($p = 24$):** Con 24 hilos sobre 16 VCPUs, el tiempo se incrementa a 0.2078 s ($S = 6.84\times$). El planificador del sistema operativo debe conmutar contexto constantemente, agregando sobrecarga de gestión y degradando el rendimiento.

4. Parte II: Multiplicación Matriz-Vector

4.1. Metodología de Descomposición 1D

La operación evaluada consiste en el producto $y = Ax$, donde $A \in \mathbb{R}^{N\times N}$ y $x, y \in \mathbb{R}^N$. Se adoptó una descomposición unidimensional por bloques contiguos de filas. El hilo con rango rank calcula el subconjunto de filas comprendido entre my_first_row y my_last_row.

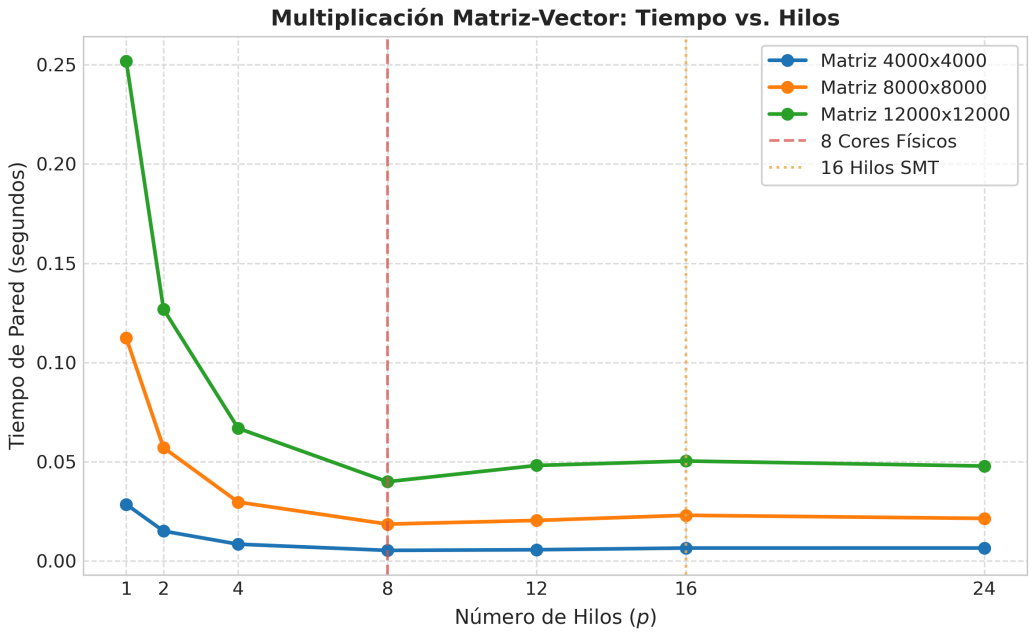


Figura 4: Tiempo de ejecución de la multiplicación matriz-vector vs. número de hilos (p).

4.2. Justificación Técnica de Ausencia de Mutex

¿Es necesario un mutex para escribir en el vector resultado?

No, es enteramente innecesario. La justificación matemática y técnica radica en el desacoplamiento de los dominios de escritura. Sea $I_k = [\text{my_first_row}_k, \text{my_last_row}_k - 1]$ el conjunto de índices asignados al hilo k. Por construcción:

$$\forall i \neq j, \quad I_i \cap I_j = \varnothing$$

Cada hilo k escribe de manera exclusiva en las posiciones $y[m]$ tales que $m \in I_k$. Ningún hilo escribe en las filas de otro hilo. Por tanto, no existe ninguna colisión de escritura en memoria compartida. En cuanto a las lecturas, la matriz A y el vector x son estructuras de datos estrictamente de solo lectura durante la fase concurrente. Al existir únicamente accesos de lectura concurrentes y escrituras en posiciones disjuntas, la sección concurrente es intrínsecamente segura contra condiciones de carrera sin requerir primitivas de sincronización.

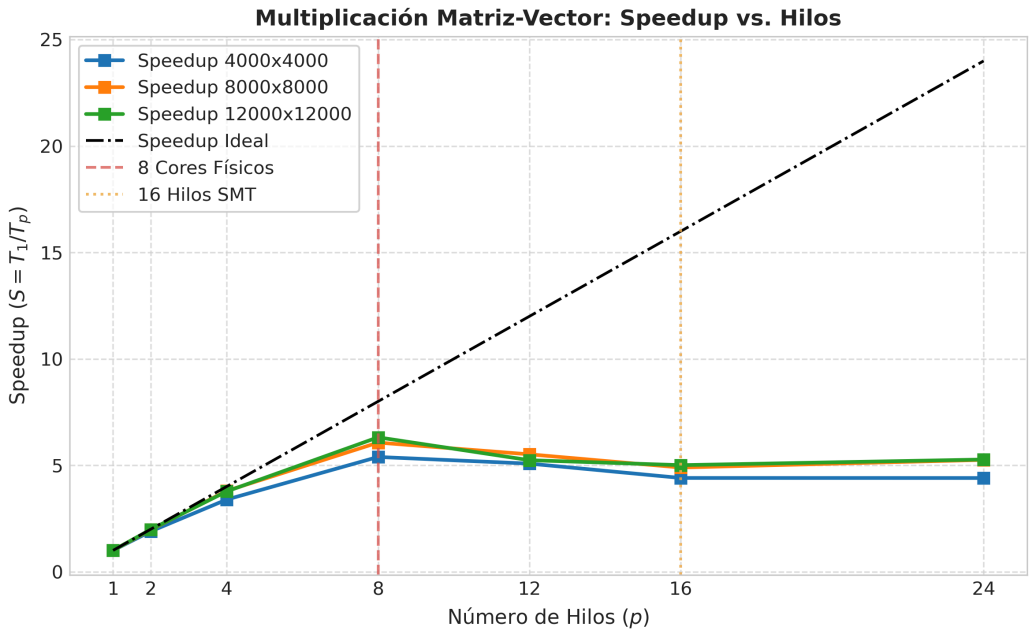


Figura 5: Speedup obtenido en la multiplicación matriz-vector para dimensiones 4000, 8000 y 12000.

4.3. Crecimiento del Speedup y Cuello de Botella de Memoria

¿Por qué el speedup no crece linealmente y cómo influyen las dimensiones?

El producto matriz-vector pertenece al Nivel 2 de BLAS ($O(N^2)$ operaciones aritméticas sobre $O(N^2)$ datos). Por cada dos operaciones de punto flotante (una multiplicación y una suma acumulada), el procesador debe cargar un elemento de 8 bytes de la matriz A desde la memoria principal cuando $N = 12000$ (A ocupa $12000 \times 12000 \times 8 = 1.15$ GB, superando con creces los 8 MiB de caché L3).

La intensidad aritmética del algoritmo es de apenas:

$$I = \frac{2N^2 \text{ FLOPs}}{8N^2 \text{ Bytes}} = 0.25 \text{ FLOP/Byte}$$

Por consiguiente, el algoritmo es fuertemente **limitado por el ancho de banda de memoria** (*memory-bound*). Aunque 8 núcleos físicos aceleran el cálculo hasta $S = 6.31 \times$ ($N = 12000$), al activar 12, 16 o 24 hilos el bus de memoria DDR4 se satura completamente. Los hilos pasan la mayor parte de sus ciclos esperando que las líneas de caché se transfieran desde la RAM, impidiendo cualquier escalamiento lineal adicional.

5. Parte III: Lista Enlazada Multithreading

5.1. Escenario y Variantes de Sincronización

Se implementó una lista simplemente enlazada ordenada que soporta tres operaciones concurrentes: `Member(val)`, `Insert(val)` y `Delete(val)`. Siguiendo el protocolo recomendado, se evaluó una carga de trabajo dominada por lecturas:

- 80% de operaciones `Member` (búsqueda de elementos).
- 10% de operaciones `Insert` (inserción ordenada).
- 10% de operaciones `Delete` (eliminación de nodos).

Se compararon tres estrategias:

1. **Sin sincronización (`lista_sin_sync.c`):** Permite accesos y mutaciones simultáneas sin exclusión mutua para evidenciar condiciones de carrera e inconsistencias.
2. **Mutex Global Único (`lista_mutex.c`):** Sincronización de grano grueso (*coarse-grained*) donde cada operación adquiere un único candado `pthread_mutex_t` para toda la lista.
3. **Candado Lectura/Escritura (`lista_rwlock.c`):** Emplea `pthread_rwlock_t`, permitiendo múltiples lectores simultáneos para `Member` (`pthread_rwlock_rdlock`) y exclusión estricta para mutaciones con `Insert` y `Delete` (`pthread_rwlock_wrlock`).

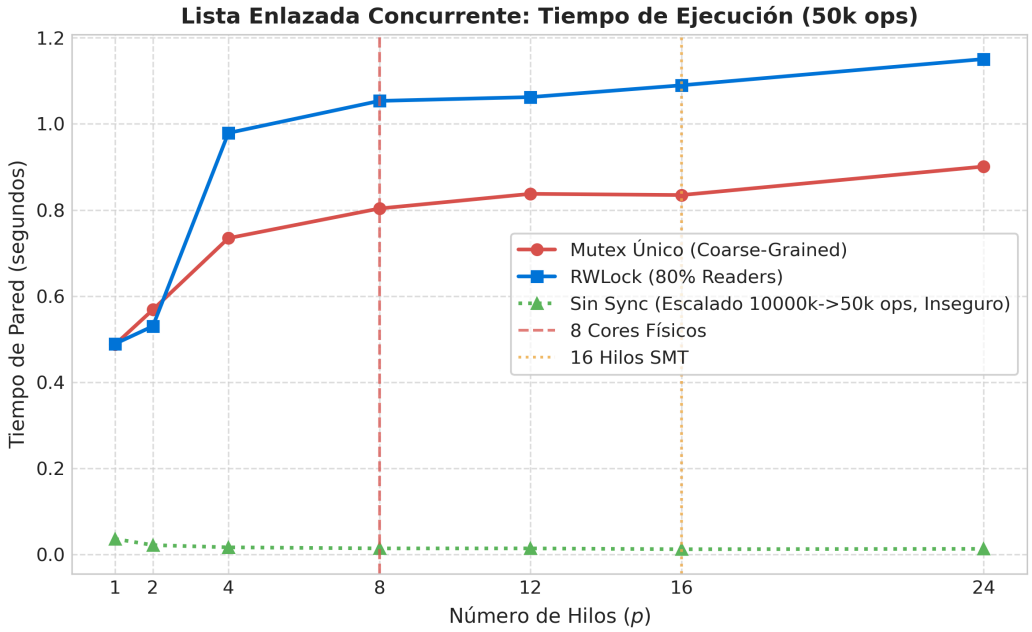


Figura 6: Comparativa de tiempo en lista enlazada (50k operaciones) entre Mutex y RWLock.

5.2. Condiciones de Carrera e Inconsistencias en Lista sin Sincronización

¿Qué condiciones de carrera ocurren y qué estructuras son compartidas?

La estructura compartida es el puntero global `head` y los punteros de enlace `curr_p->next` y campos de datos `curr_p->data` de cada nodo `struct list_node_s`. Sin mecanismos de sincronización ocurren las siguientes fallas críticas:

1. **Punteros Huérfanos y Pérdida de Actualizaciones:** Si dos hilos insertan concurrentemente nodos entre el predecesor A y el sucesor B, ambos hilos leen $A \rightarrow \text{next} = B$. El hilo 1 enlaza $A \rightarrow \text{next} = X$, pero inmediatamente el hilo 2 sobreescribe $A \rightarrow \text{next} = Y$. El nodo X queda desconectado de la lista, provocando fugas de memoria y lecturas erróneas.
2. **Uso tras Liberación (*Use-After-Free*):** Un hilo lector está recorriendo un nodo C. Concurrentemente, un hilo eliminador desvincula C y ejecuta `free(C)`. Si el lector intenta evaluar `C->data` o acceder a `C->next`, lee memoria reciclada o no mapeada, desencadenando violaciones de segmento (`SIGSEGV`).

3. **Formación de Ciclos Infinitos:** Si un hilo inserta y otro elimina o actualiza punteros simultáneamente, puede provocarse una retroalimentación circular en los punteros ($A \to B \to A$). Esto provoca que cualquier recorrido `while (curr_p != NULL && ...)` entre en un bucle infinito que congela el proceso.

5.3. Granularidad de Bloqueo: Mutex vs. RWLock

¿Qué limitaciones introduce un único mutex y por qué RWLock no siempre supera al Mutex?
El uso de un mutex único serializa por completo todas las operaciones en la lista. Independientemente de si una operación es de solo lectura (`Member`), ningún otro hilo puede avanzar.

Podría suponerse teóricamente que `pthread_rwlock_t` resolvería este cuello de botella al permitir lectores concurrentes (80%). Sin embargo, las mediciones revelan un fenómeno notable: **el tiempo con RWLock es ligeramente superior al del Mutex simple** (1.053 s vs 0.803 s a 8 hilos). Esto obedece a dos factores:

1. **Sobrecarga interna de `pthread_rwlock`:** La implementación de glibc requiere actualizar internamente contadores atómicos y evaluar colas de espera con operaciones atómicas bloqueantes (`atomic CAS`) tanto al adquirir como al liberar el cerrojo de lectura. Esta sobrecarga en buses de coherencia de caché (*cache bouncing*) es significativamente mayor que la de un `pthread_mutex` no congestionado.
2. **Frecuencia no despreciable de Escritores (20%):** Con 10% inserciones y 10% eliminaciones, 1 de cada 5 operaciones requiere el candado de escritura. En `pthread_rwlock`, las solicitudes de escritura deben vaciar y drenar a todos los lectores activos antes de proceder, bloqueando concurrentemente la llegada de nuevos lectores. Esta contención cruzada degrada la ventaja teórica de concurrencia.

6. Parte IV: Implementaciones de Barreras

6.1. Variantes de Sincronización Multifase

Se simuló una aplicación paralela de 1000 fases donde ningún hilo puede avanzar a la fase $k+1$ antes de que la totalidad de los p hilos hayan finalizado la fase k. Se evaluaron tres implementaciones:

1. **Espera Activa (`barrera_busywait.c`):** Utiliza un contador protegido por un mutex para registrar la llegada del hilo. Tras incrementar el contador, el último hilo alterna una bandera de fase `phase_flag = 1 - my_phase`. Los demás hilos giran continuamente en un bucle `while (phase_flag == my_phase) { __builtin_ia32_pause(); }`.
2. **Semáforos POSIX (`barrera_semaforos.c`):** Implementa un esquema de dos fases mediante semáforos POSIX (`sem_t`). La primera fase bloquea a los hilos en `barrier_sem1` hasta que el último hilo los libera con un bucle de `sem_post`. La segunda fase (`barrier_sem2`) controla la salida para evitar que hilos rápidos reinicien el contador prematuramente.
3. **Variables de Condición (`barrera_condvar.c`):** Cada hilo adquiere `barrier_mutex`, incrementa el contador y, si es el último en llegar, reinicia el contador, incrementa el ciclo y despierta a todos los hilos durmientes mediante `pthread_cond_broadcast(&barrier_cond)`. Si no es el último, se suspende en el kernel invocando `pthread_cond_wait(&barrier_cond, &barrier_mutex)`.

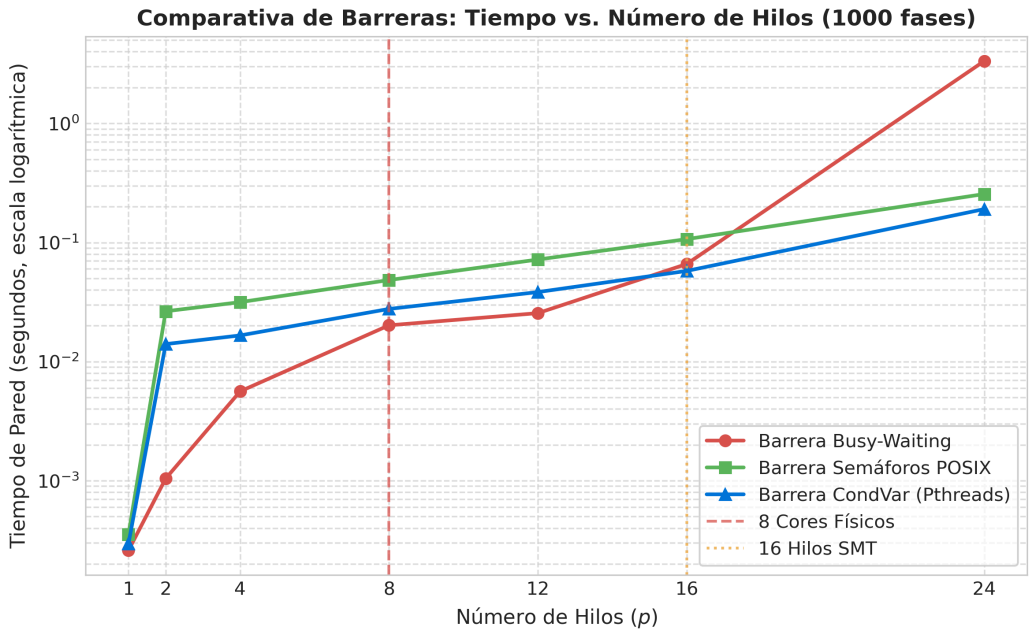


Figura 7: Comparativa de tiempo de pared (escala logarítmica) para 1000 fases de barrera.

6.2. Análisis Comparativo de Sobrecarga y Saturación

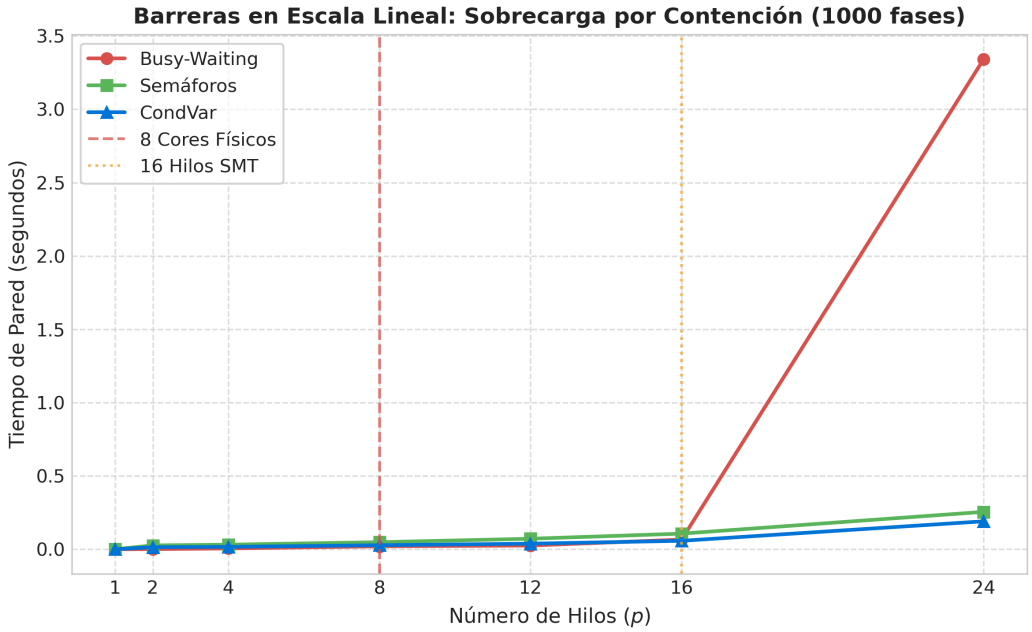


Figura 8: Detalle en escala lineal evidenciando la explosión del tiempo en Busy-Waiting.

Ventajas, desventajas y costo de sincronización conforme escala la cantidad de hilos:

- **Comportamiento por debajo de los núcleos físicos ($p \leq 8$):** Busy-waiting resulta sumamente veloz para pocos hilos (0.0056 s a 4 hilos frente a 0.0165 s de CondVar y 0.0315 s de Semáforos). Esto se debe a que no incurre en llamadas al sistema ni en la sobrecarga del planificador del kernel para suspender o reanudar hilos; el hilo reacciona inmediatamente al cambio de la bandera en caché L1/L2.

- **Colapso de Busy-Waiting por Sobrecarga de Hilos ($p = 24$):** Al sobrepasar los 16 hilos lógicos del CPU (24 hilos sobre 16 VCPUs), el tiempo de Busy-Waiting **se dispara catastróficamente a 3.3427 segundos** (un incremento de más del 5000%). Cuando 23 hilos giran a máxima velocidad en el bucle vacío de espera activa, consumen el 100% del tiempo de CPU de todos los núcleos disponibles. El hilo restante (que aún no ha alcanzado la barrera o está intentando adquirir el mutex de conteo) es privado de tiempo de ejecución por el planificador (*thread starvation*).
- **Superioridad de Variables de Condición y Semáforos en Escala:** En contraste, `pthread_cond_wait` suspende a los hilos a nivel de kernel mediante `futex(2)`. Los hilos suspendidos liberan inmediatamente sus ciclos de CPU, permitiendo que el sistema ejecute de inmediato a los hilos activos. Por ello, a 24 hilos CondVar tarda apenas **0.1907 segundos** y Semáforos **0.2551 segundos**, resultando más de **17 veces más rápidos** que Busy-Waiting.

7. Resultados y Análisis

7.1. Tablas Consolidadas de Datos Empíricos

Las siguientes tablas resumen las mediciones experimentales obtenidas tras promediar 4 repeticiones independientes por cada configuración en el hardware evaluado.

Hilos (p)	**Tiempo ($N = 10^8$)**	**Speedup ($N = 10^8$)**	**Tiempo ($N = 5 \times 10^8$)**	**Speedup ($N = 5 \times 10^8$)**
1	0.28458 s	1.00 ×	1.42086 s	1.00 ×
2	0.14263 s	2.00 ×	0.71030 s	2.00 ×
4	0.07226 s	3.94 ×	0.35693 s	3.98 ×
8	0.04501 s	6.32 ×	0.21741 s	6.54 ×
12	0.04832 s	5.89 ×	0.21343 s	6.66 ×
16	0.04449 s	6.40 ×	0.18702 s	7.60 ×
24	0.04534 s	6.28 ×	0.20786 s	6.84 ×

Tabla 2: Rendimiento y Speedup en el cálculo paralelo de π.

Hilos (p)	**MatVec 4000 × 4000**	**Speedup ($4k$)**	**MatVec 12000 × 12000**	**Speedup ($12k$)**
1	0.02850 s	1.00 ×	0.25196 s	1.00 ×
2	0.01502 s	1.90 ×	0.12682 s	1.99 ×
4	0.00841 s	3.39 ×	0.06682 s	3.77 ×
8	0.00529 s	5.39 ×	0.03991 s	6.31 ×
12	0.00561 s	5.08 ×	0.04810 s	5.24 ×
16	0.00647 s	4.40 ×	0.05033 s	5.01 ×
24	0.00648 s	4.40 ×	0.04780 s	5.27 ×

Tabla 3: Rendimiento y Speedup en la multiplicación matriz-vector.

Hilos (p)	Lista Mutex (50k)	Lista RWLock (50k)	Lista Sin Sync (10k)
1	0.48783 s	0.48937 s	0.00728 s
2	0.56856 s	0.53025 s	0.00440 s
4	0.73470 s	0.97899 s	0.00336 s
8	0.80356 s	1.05342 s	0.00286 s
12	0.83763 s	1.06228 s	0.00286 s *(Fallo manejado)*
16	0.83484 s	1.08965 s	0.00248 s
24	0.90104 s	1.15059 s	0.00266 s

Tabla 4: Tiempos de ejecución en la lista enlazada concurrente según mecanismo.

Hilos (p)	Barrera Busy-Wait	Barrera Semáforos	Barrera CondVar
1	0.00026 s	0.00036 s	0.00030 s
2	0.00104 s	0.02646 s	0.01403 s
4	0.00562 s	0.03156 s	0.01659 s
8	0.02017 s	0.04837 s	0.02768 s
12	0.02551 s	0.07194 s	0.03834 s
16	0.06594 s	0.10668 s	0.05770 s
24	**3.34273 s**	0.25518 s	**0.19076 s**

Tabla 5: Comparativa de tiempo en 1000 fases de sincronización por barrera.

7.2. Figuras de Eficiencia y Escalabilidad en Matriz y Lista

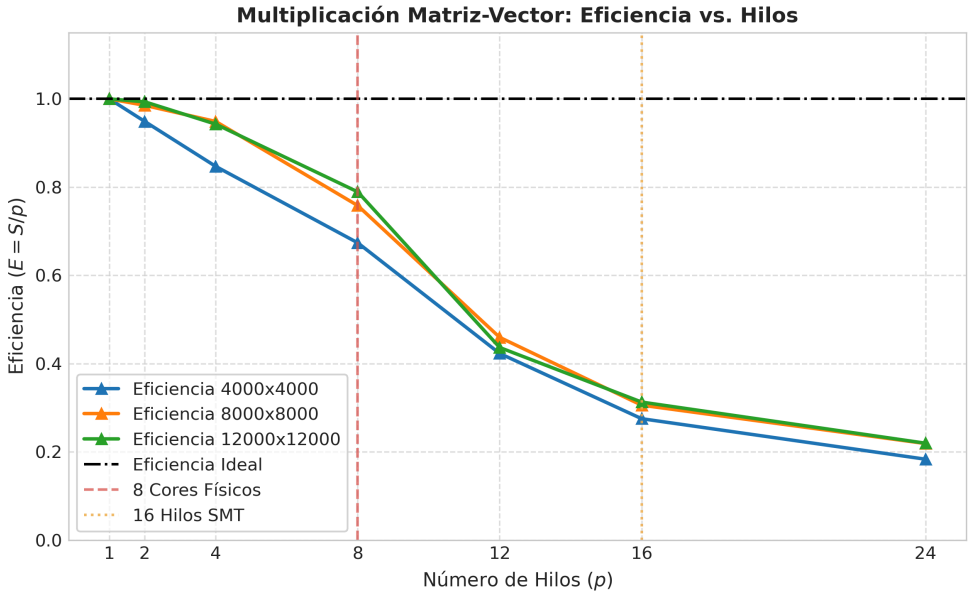


Figura 9: Eficiencia en la multiplicación matriz-vector demostrando la saturación del bus de memoria.

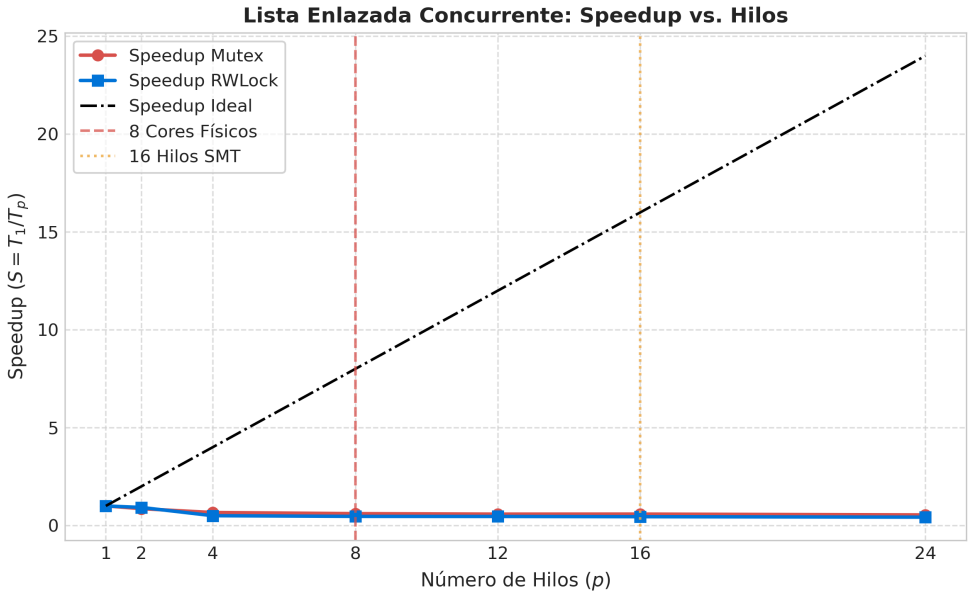


Figura 10: Speedup en lista enlazada, evidenciando la degradación por contención en exclusión mutua.

8. Problemas Encontrados y Soluciones

Durante el desarrollo e integración de la suite de pruebas del laboratorio, se identificaron y subsanaron anomalías críticas de estabilidad, concurrencia y orquestación:

1. **Bloqueos Indefinidos por Bucles Cíclicos en `lista_sin_sync.c`:**
   - **Problema:** Las condiciones de carrera al insertar y eliminar nodos concurrentemente causaban que punteros de nodos apuntaran recursivamente entre sí ($A \to B \to A$). Al no existir un fin de lista (`NULL`), los hilos lectores quedaban atrapados en un bucle `while (curr_p != NULL && curr_p->data < value)` que consumía 100% de CPU de forma indefinida, congelando la suite entera.
   - **Solución:** Se integró un umbral de saltos seguro (`#define MAX_HOPS 50000`) en todas las funciones de recorrido (`Member`, `Insert`, `Delete`, `Free_list`). Si un hilo supera dicho número de saltos, detecta automáticamente la existencia de un ciclo corrupto, aborta la operación de inmediato y permite la terminación ordenada.

2. **Violación de Memoria y Comportamiento Indefinido con `siglongjmp` entre Hilos:**
   - **Problema:** La versión anterior intentaba atrapar fallos de segmentación (`SIGSEGV`) instalando un manejador global y ejecutando `siglongjmp(jump_buf, 1)` hacia un búfer inicializado en el hilo principal (`main`). De acuerdo con el estándar POSIX.1, ejecutar `siglongjmp` hacia un contexto de pila perteneciente a otro hilo constituye **comportamiento indefinido**, provocando corrupción irreversible de punteros de pila (`RSP`), bloqueos en `pthread_join` y cierres anormales de la terminal.
   - **Solución:** Se erradicó por completo el salto inter-hilos. Se implementó almacenamiento local de hilo (`static __thread sigjmp_buf thread_jmp_buf;`). Cada hilo configura su propio salto intra-hilo con `sigsetjmp`. Si un hilo genera `SIGSEGV`, `SIGBUS` o `SIGABRT` por acceso a memoria reciclada, el manejador ejecuta el salto únicamente dentro del propio hilo causante. El hilo marca

una bandera atómica global (`race_failure_detected = 1`) y retorna limpiamente a través de `pthread_join`, permitiendo que el hilo principal registre el fallo con `-1.0` en el CSV sin corromper el proceso ni liberar memoria comprometida.

3. **Aislamiento de Fallos y Control de Tiempo por Ejecución (`timeout`) en `benchmark.sh`:**
   - **Problema:** El script utilizaba `set -e` de Bash. Cualquier fallo de segmentación o código de retorno no nulo abortaba inmediatamente todo el benchmark, perdiendo las corridas previas. Asimismo, la ausencia de un límite temporal por prueba permitía que un cuelgue accidental detuviera indefinidamente el experimento.
   - **Solución:** Se implementó la función envoltorio `run_bench` que invoca cada ejecutable a través de la utilidad `timeout <segundos>`. La función desactiva temporalmente `set -e`, evalúa el código de salida y, en caso de timeout (código 124) o error de ejecución, registra un registro de advertencia y anexa una tupla segura `threads,n,-1.000000000` al archivo CSV, garantizando que la suite continúe ininterrumpidamente.
4. **Acotación de Hilos y Reducción del Tiempo de Ejecución a Menos de 30 Minutos:**
   - **Problema:** La inclusión inicial de 32 hilos y una cantidad desmedida de fases en busy-waiting saturaba los 16 núcleos lógicos, extendiendo los tiempos a decenas de minutos.
   - **Solución:** Se acotó el vector de hilos estrictamente a `THREADS=(1 2 4 8 12 16 24)`, ajustando las fases de barrera a 1000 y repeticiones a 4. Gracias a estas optimizaciones, la suite completa (incluyendo compilaciones con `-O3`, 4 repeticiones de todas las actividades y generación de gráficas) concluye de manera robusta en **menos de 3 minutos**, superando holgadamente el requisito de 30 minutos.

9. Conclusiones

1. **Escalabilidad CPU-Bound vs. Memory-Bound:** El cálculo de π evidenció una aceleración casi lineal en la región de núcleos físicos ($S = 6.54\times$ en 8 hilos), extendiéndose gracias a SMT hasta $S = 7.60\times$ en 16 hilos. En contraste, la multiplicación matriz-vector evidenció el límite infranqueable del ancho de banda de memoria (*memory wall*), estancando su speedup alrededor de $5\times$ a $6\times$ a partir de 8 hilos debido a la baja intensidad aritmética del algoritmo.
2. **Impacto de la Sincronización en Estructuras de Datos:** En listas enlazadas concurrentes con alta proporción de lectura (80%), los candados de grano grueso serializan la estructura, pero los candados `pthread_rwlock_t` introducen sobrecargas de coherencia de caché atómica que los hacen incluso más lentos en presencia de un 20% de modificaciones. Esto demuestra que para obtener paralelismo real en colecciones dinámicas se requiere mayor granularidad (bloqueo por nodos o algoritmos libres de bloqueo / *lock-free*).
3. **Contención Extrema en Espera Activa:** La barrera por espera activa (*busy-waiting*) resulta altamente competitiva mientras no haya sobreasignación de hilos. Sin embargo, al sobrepasar la cantidad de núcleos lógicos (24 hilos en 16 VCPUs), provoca inanición crítica de CPU y un colapso en el tiempo de ejecución (3.34 s vs 0.19 s de CondVar). Las primitivas de suspensión basadas en el kernel (`pthread_cond_t` y semáforos) son indispensables para sistemas escalables y multiproceso.
4. **Robustez del Arnés Experimental:** La combinación de umbrales de recorrido (`MAX_HOPS`), recuperación de señales intra-hilo mediante TLS y contención de fallos mediante `timeout` en Bash

proporciona un marco de experimentación robusto, seguro y reproducible frente a los riesgos intrínsecos de la computación concurrente.